Smart Accident Monitoring System (SAMS)

Technical Report

Author: UTERAMAHORO Avellin Bonaparte

Repository: Link

Video: Link

Overview

SAMS is an on-site MVP that fuses phone-camera vision and machine accelerometer/ToF sensing into a live risk label (**NORMAL / CAUTION / DANGER**), drives Arduino actuators, archives sessions, and supports an MLOps review/retrain loop. This report maps delivery to the approved capstone scope, summarises technical behaviour and testing, and documents code quality, OOP and algorithmic choices. The server runs on a laptop on the local workshop network with **PostgreSQL** and media/ for video storage.

1. Scope Alignment & Feature Validation

Features below are mapped to the approved proposal milestones (edge sensing, multi-model risk decision, operator dashboard, and a path to retrain from site data). Status uses Completed / Partial / Not implemented. Evidence points at repository paths and UI screens under Design/.

Planned Feature	Intended Purpose	Status	Evidence
Worker detection & tracking	Know who is near the machine	Completed	vision_inference.py, person_tracker.py (ByteTrack); Live overlays
Worker distance & zones	Place each worker in caution/danger zones	Completed	distance_estimation.py, hardware/tof.py, notebook 04; results/vision/distance/
Worker motion / approach	Detect rushing toward the machine	Completed	person_tracker.py MotionLog; fusion rush-in scenario checks
PPE compliance	Flag missing hard hat, vest, gloves, boots, goggles	Partial	ppe_inference.py, notebook 05; test missing-gear recall still low
Fire & smoke	Detect scene hazards	Completed	hazard_inference.py, notebook 06; mAP50-95 0.394
Fall detection	Detect an injured / lying worker	Partial	accident_inference.py, notebook 07; mainly synthetic falls
Safe / unsafe behaviour	Classify workplace behaviour on live frames	Completed	behaviour_inference.py; ResNet18 fine-tune served live; notebook 01
Machine state	Know whether the machine is running	Completed	machine_state_inference.py, notebook 02 §8, stead_dataset/
Vibration anomaly	Detect abnormal machine vibration	Partial	vibration_inference.py, notebook 02; UCI SVM — weak board transfer
Fusion risk decision	One NORMAL / CAUTION /	Completed	control_center.py, fusion_inference.py,

	DANGER output		notebook 03; Live page
On-site alerts	Warn with buzzer / LEDs in real time	Completed	hardware/, firmware/, Design/Schematic-Design.png
Dashboard, archive, alerts	Review what happened and why	Completed	review_views.py, Design/Homepage.png, Archive, Alerts
Video upload analysis	Analyse footage without the board	Completed	video_analysis.py, Celery tasks; Archive → Upload
MLOps retraining loop	Learn from the site's own labelled data	Completed	mlops.py, trials.py; MLOps Keep/Exclude + trial export

Justified scope changes.

(1) *Machine state* was added: vibration alone cannot tell a stopped machine from a quiet healthy one.

(2) Fusion grew from ~10 to **14 inputs** so PPE, fire, smoke and falls can change the risk label, not only decorate the UI.

(3) Partial rows are **data-limited**, not code-limited, thresholds/classes come from each model's metadata.json, so site retraining is a drop-in.

(4) Behaviour is served live (ResNet18 fine-tune) but remains domain-limited to public-facility training data until archive labels grow.

2. Technical Functionality

2.1 Architecture and backend / frontend / data

Edge. An Android phone scans the Go Live QR code and POSTs JPEG frames over HTTPS to /cam/<token>/frame/ (HTTPS required for getUserMedia). An Arduino streams ax,ay,az at 50 Hz and TOF:<m> over USB serial (115200 baud) and accepts buzzer/LED/motor commands with # ACK.

Server. Django 6.1 on uvicorn (ASGI). AccelStream buffers accelerometer samples; each model gets its own window length from metadata. The Live page polls /api/control/live/ (~0.4 s), which runs vision, reads the board, runs fusion, drives actuators, and returns JSON for the UI.

Data. PostgreSQL via the Django ORM: Recording / RecordingMinute (archive cards with every model output and fusion inputs), InteractionEvent (decision audit), Trial / TrainingSample (labelled collection). Videos and frames live under media/. Upload analysis and live finalize run on Celery (Redis) with a thread fallback.

2.2 Decision logic

Per frame: PPE finds people and gear; gear/fall boxes assign to the person box containing their centre; pose places joints; the person in the ToF beam gets ToF distance inside the calibrated near range, others get the residual distance network ($Z = Z \cdot e^{\delta}$); speed and approach are the least-squares slope of tracked distances over the fusion model's motion window. Machine-state and

vibration label the latest accel window. These 14 values form the fusion vector. **If any required input was not measured, there is no decision** and the UI names what is missing, nothing is silently imputed. Empty frames use the model's absent_worker values. On a label change, buzzer/LEDs follow and an InteractionEvent is logged.

2.3 Error handling and responses to different inputs

Input / condition	System response
No Arduino connected	Vision still runs; machine/vibration offline; fusion → "No decision · not measured …"
Model weights / metadata missing	That model reports unavailable; server keeps running; fusion waits on missing inputs
Empty frame	Absent-worker person inputs → typically NORMAL unless fire/smoke
Worker seen for one frame only	Motion not measurable → no decision until two distances exist
Bad sensor recording	Recorder rejects constant / not ≈1 g / not ≈50 Hz runs
Wrong sign-in credentials	Error alert on login; dashboard routes require authentication
Uploaded video (no board)	Same pipeline; operator-declared machine/vibration marked "declared" in archive/export

3. Testing, Validation & Performance

At least two strategies are demonstrated below (unit + integration), plus model validation, hardware checks and a reproducible performance benchmark. Walkthrough video shows selected cases from §2.3 and these strategies.

3.1 Unit testing

Why: decision joins must be exact, a wrong assignment silently changes risk.

How: python manage.py test monitoring with mocked models; modules test_vision, test_fusion, test_board, test_archive, test_model_loader, test_behaviour, test_celery. Covers PPE thresholds, centre-in-box assignment, ToF beam, motion slopes, vibration features, fusion assembly, loader concurrency, MLOps keep/export.

Result: 57+ tests passing.

Improvements after: per-class PPE thresholds; metadata-driven windows; LazyModel consolidation; fusion fixtures updated for the 14-input contract.

3.2 Integration testing

Why: archive, trials and export cross DB, files and services.

How: Django TestCase: live session → recording + cards; upload → fusion with declared inputs; trials keep rows only when every fusion input is measured; MLOps export columns match notebooks 03/04. **Result:** all pass.

Improvements after: rush-in scenario added so all 15 fusion checks pass; trial start ends any running

trial.

3.3 Model validation (ML track)

Every notebook splits by recording/trial/video/subject, selects on validation, reports test once. Fusion also uses 15 hand-written scenarios; machine state uses leave-one-recording-out.

Model	Approach	Headline test result
Fusion	Random Forest, 14 inputs	DANGER recall 1.0; macro-F1 0.997; 15/15 checks
Machine state	Random Forest on board windows	STOPPED/RUNNING recall 1.0 (held-out)
Vibration	SVM-RBF, 48 features (UCI)	Macro-F1 0.80 (val 0.86); board transfer weak
Distance	Pinhole × residual MLP	MAE 0.22 m vs pinhole 0.79 m (unseen person)
Pose	YOLOv8s-pose fine-tuned	PCK 0.854 vs stock n-pose 0.767
Fire/smoke	YOLOv8s @ 960 fine-tuned	mAP50-95 0.394; fire frame recall high on val
PPE	YOLOv8n @ 640 fine-tuned	mAP50-95 0.235 (val 0.333); missing-gear weak
Fall	YOLOv8s fine-tuned	17/17 falls on synthetic test; FP on real scenes open
Behaviour	ResNet18 ImageNet fine-tune	Binary acc. 0.76; unsafe recall 0.85 (live)

3.4 Hardware / device testing

Why: sensor faults look like valid data.

How: hw.sh check; firmware WHO_AM_I; recording quality gates.

Result: stopped + motor low/high/intermittent recordings accepted and used for machine-state training.

3.5 Performance & compatibility

python manage.py benchmark on a 1280×720 frame (10 timed runs after warm-up). Host: Intel Core i5-8259U, 8 GB RAM, CPU-only, macOS 15, Python 3.12, PyTorch 2.2.2.

Stage	Median	Notes
PPE + person (YOLOv8n 640)	76 ms	12% of full frame
Pose (YOLOv8s-pose)	169 ms	27%
Fire/smoke (YOLOv8s 960)	289 ms	Bottleneck ~45%
Fall (YOLOv8s)	153 ms	24%
Full vision + distance	636 ms ≈ 1.6 FPS	Peak RAM ≈ 710 MB
Fusion RF	50 ms	After vision
Machine state / vibration	46 / 6 ms	Per ~2 s window

Analysis. ~0.64 s/decision on this CPU means the 0.4 s poll is effectively ~1.5 decisions/s, adequate for walking approaches, not sprint rushes. Planned: run hazard every second frame, ONNX/OpenVINO, or a GPU edge box.

Environment	Configuration	Result
Server OS / runtime	macOS 15 (Intel), Python 3.12, Django 6.1, uvicorn, PostgreSQL	Works
Python 3.13+	No wheels for torch 2.2.2 / numpy 1.26.4	Not supported (documented)
Training	Google Colab GPU, Ultralytics 8.3	Notebooks 01–08 end-to-end
MCU / sensors	Arduino Uno + MPU6500 + VL53L0X	Works; 6050/9250/9255 detected
Phone camera A	Pixel 7 Pro, Android 17, Chrome	Works (HTTPS stream)
Phone camera B	Galaxy M11, Android 12, Chrome	Works
Dashboard browsers	Chrome and Safari on macOS	Works

4. Code Quality & Technical Implementation

4.1 OOP and structure

- **Separation of concerns:** SAMS/ config; monitoring/ app; monitoring/services/ (one module per model + fusion/tracking/MLOps); hardware/ drivers; firmware; notebooks; results. Views call services; services never render HTML.

- **Abstraction / inheritance / polymorphism:** abstract LazyModel to JoblibModel, YoloModel, PoseModel, DistanceNet (shared get/reset); abstract AccelSource to SerialMPU6050; AccelStream depends only on the interface.

- **Encapsulation:** Buzzer, Leds, Motor, ToFSensor, PhoneCamera, MotionLog, LiveRecorder own state behind small APIs.

- **Open for extension:** each model's metadata.json carries classes, thresholds, windows and input order so retrained weights are drop-in.

- **Thread safety:** double-checked locking in LazyModel (tested under concurrent load).

4.2 Algorithms and problem-solving

Problem	Solution
Same worker across frames	ByteTrack IDs from the PPE detector
Which gear belongs to whom	Assign gear/fall box to person box containing its centre
Speed without a dedicated speed sensor	np.polyfit slope over a bounded distance history
Monocular distance	Pinhole Z_0 x learned residual MLP (Huber on log-space)
Fuse ToF + vision	ToF only if person is in beam and within near range
Rare dangers vs false alarms	Per-class thresholds; fusion selected on DANGER recall
Vibration from raw IMU	48 time- and frequency-domain features per window
Honest metrics	Leakage-free splits; scenario + leave-one-recording-out checks

4.3 Quality practices

1. Consistent snake_case naming
2. type hints/docstrings in services
3. env-based configuration
4. tests grouped by area with shared fixtures
5. reproducible benchmark command
6. README + hardware README with clear instructions